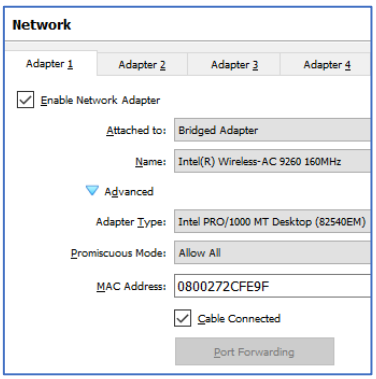
SER335 Lab 5

Objectives:

1. Conduct an SQL Injection Attack
2. Apply Exception Handling Practices and Defensive Programming

Activity 1: SQL Injection Attack (50 points)



Environment Setup:

Setup the two environments:

1. In SEED VM: Badstore (deliberately insecure web application) docker container
   a. Make sure the SEED VM is using the "NAT Network" adapter under Network Settings
   b. Download the gzipped tarball file from the link on Canvas
   c. Copy the gzipped tarball into the VM
   d. In the VM: Unzip the file (gunzip badstore.tar.gz)
   e. In the VM: Load the docker image: "docker load -i badstore.tar"
   f. In the VM: Docker images → you should see "badstore"
   g. In the VM: Start a container from this image: docker run -d –-net=host <image>
2. Kali Linux (Linux System to attack Badstore Web application)
   a. Setup Network using the "NAT Network" adapter under Network Settings
3. Check the current network settings of the Kali and Badstore machine using the ifconfig command: *ifconfig*
   Both systems should be on the same network i.e. both should have netmask 255.255.255.0 and the first three numbers of the IP address should be the same (10.0.2.x).

4. In Kali: Open the Badstore site by entering the IP address of the Badstore VM in a browser. e.g. http://10.0.2.7 (note the BadStore IP will vary and be based on the output of your ifconfig command above). You will attack from Kali to Badstore (SEED) VM

   Navigate the website and explore for vulnerabilities. The login credentials of a supplier are:
   Username: *big@spender.com* Password: *money*

Task 1: SQL Injection (15 points)

Bypass the authentication of a vulnerable login page in Badstore using SQL injection.

1) Enter some appropriate syntax to modify the SQL query into the "*Email Address*" field and login into the website as a Master System Administrator (Hint: See Dr. Findler's lecture on SQL Injection Attacks). Take a screenshot capturing what you entered into the Email Address field.
2) View all of Joe's previous orders, and take a screenshot of that webpage.
3) Find the secret admin portal by modifying the query string parameter in your browser (the query string is the part after the ? in the URL, e.g. *?action=login*. We are asking you to find a value for "action" that gets you on the admin portal. It is not hard for an *admin* to guess it. Take a screenshot of that webpage and be sure to include the URL in the address bar.

Task 2: Brute Force Attack on the Underlying Database (35 points)

Assume the Badstore developers close that gaping vulnerability in Task 1. You seek a way to get into the database that stores the information for the site. Perform a pen test of brute-forcing the DB credentials and getting access to the database. Do the following:

1) In Kali, discover if the database port is open or not, and what it is. Provide a screenshot of the command you run and highlight the open database port you find. *Hint*: The database is mysql but the port has been changed from the default port number to something close to it. You have to figure out what it is.
2) Try all the combinations of usernames and passwords to access the database (brute force). Use the *'scanner/mysql/mysql_login'* exploit of the *Metasploit* tool to perform the attack.
   a. You will need to set the RHOSTS and RPORT options properly. See '*show options*' in metasploit
   b. The list of possible usernames and passwords can be found in *'/usr/share/wordlists/metasploit/root_userpass.txt*. Create a screenshot of the Metasploit tool highlighting the discovered database credentials.
3) Login to the target database from a kali terminal using the above-discovered credentials.
   Command: `mysql -h ip_address -u user_name -P port -p badstoredb`

   List all the users and their passwords in the database and provide a screenshot of the same:

```
mysql-> show tables;
mysql-> select * from userdb;
```

4) The passwords are all hashed, use this tool to decrypt the passwords. What is the username and password of the System Administrator "admin" user? Provide a screenshot of the tool output.
5) Create a new user with admin privileges by inserting a new row in the user details table. Using this tool to generate hashed passwords. Submit a screenshot highlighting the query and the newly created user.

Each step is worth 7 points.

Submission for Activity 1:

Create a document <asurite>_lab5_SQLInjectionActivity.[doc|docx|pdf] based on the screenshots requested, commands entered, and questions asked throughout this activity. Clearly label each part of the document according to the Task and step number within the task.

Note: You can google up the badstore application, and even get a user's manual. It has been around a long time. However, we have modified most aspects of the badstore application in such a way that the exploits you find on the web won't work the same way.

Activity 2: Exception Handling and Defensive Programming (50 points)

Task 1: Exception Handling (30 points)

In our never-ending quest to make JFM more secure, we looked at how JFM handles exceptions. We found:

1. There are 60 instances of "Exception.printStackTrace()". In some files the swallowing of exceptions is so egregious that the exception variable name is literally "ignored"! (for example, org/jfm/filesystems/JFMFileSystem.java)
2. There are many uses of unchecked exceptions that may not be caught
3. There are many throws/rethrows of a new exception (throw new …). This is not altogether always bad but would be a place to look for poor exception practices
4. There are several places where exceptions are used instead of writing defensive code to avoid the exception (lazy condition checking practices)
5. There are several instances of "overzealous try blocks"

For this activity, you will fix a specific poor exception handling practice in JFM. When the application goes to add a new user (the "AddUser" functionality for admins), it must write new entries into the salts.json and authentication.json files. If these actions fail (throw an Exception), how does JFM handle it? How can it be improved? To test, startup JFM and then change the permissions of salts.json and authentication.json under the jfm-0.9.1/resources directory to remove write permissions. Then try to add a new user – what happens? What do you see in the command-line where you started up JFM?

Your task is to find the relevant code where these files are written in the call chain for adding a new user. Refactor these blocks of code such that (10 points each):

1. A more specific error message can be given in the application. Provide this message to the end user as a GUI popup dialog (JOptionPane is the simplest way to do this in Java Swing).
2. A stack trace is not printed out to stdout (the terminal window).
3. The handling of exceptions is as specific as it should be according to best practices (HINT: these blocks are examples of #5 "overzealous try blocks" indicated above).

Task 2: Apply Defensive Programming (20 points)

There are a few places where JFM lazily uses exceptions instead of using Defensive Programming to check for invalid input or parameters (#4 above). These are not going to cause Buffer Overflow or Injection Attacks, but they can be used to circumvent basic security concerns. For this task find two instances (other than the exceptions in Task 1) of this case that you consider significant errors and possible security risks, and implement Defensive Programming to rectify the concerns, which should cause the try/catch block to be removed. *Note:* Not all exception handling is bad and not all should be replaced with Defensive Programming techniques. Further, not all exceptions are in significant situations, so *think* about the places you choose to refactor (8 points each for two instances corrected).

Submission for Activity 2:

For this Activity use the version of JFM in the given code of Lab 2 (not your modified version).

Please provide a README.txt in the root of the source tree that lists exactly where you made code changes (which files) and WHY you made the change you did. Please prefix places IN THE CODE IN COMMENTS where you refactored by inserting "SER335 LAB5". We will search for these strings to find the locations you modified. If we cannot find the locations we cannot grade it and will not accept grade appeals where you ask us to regrade because you forgot to tell us where to look!